Brief: sections 2 and 3 (main body), plus the appendix

Main body budget: your section 1 is 403 words; these two add about 593, so the brief lands near 996 words, about 1.7 pages. Everything else moves to the appendix.

The findings shaped the data model, and how the design surfaced the errors

Confidence could never be the proof. Every confident mistake I found looked right on the surface and was still wrong, and 25 of the 27 were read at 95% confidence or more. That told me every number in Taxmark had to carry its own source, how it was built, how it was checked, and what it still fails.

One packet, read three ways. The same input packet was read by three readers, each with its own job, and that separation is what lets me trust the result (Appendix B). An independent, AI-assisted investigation outside the app produced my answer key of 27 mistakes, which never creates a finding and only tests the checks and grades the AI. Inside the app, 15 families of deterministic checks flag all 27, and for mistakes that live in wording an AI reader suggests issues with quotes that code verifies word for word, matching 13 of the 27 and one I had missed (Appendix D). The packet stays read-only, and everything people do is an append-only event on top of it, which makes the audit trail the data itself.

How the design surfaced the errors. The one I would show anyone is California's estimated payments. The return claims 14,000 while the FTB's record shows 10,500, because the engine counted a payment by its date and missed that it was marked "tax year 2024". The page was read at 0.98 to 0.99 confidence, so nothing looked wrong until a second source disagreed. On the Checks screen this is a single row carrying the Blocker, ✦ AI and Confident mistake pills. From there, one click opens the source page, the fix flows through to the California refund, which drops from 8,256 to 4,756, and the partner signs off on the new number. All 27 mistakes are listed in Appendix E.

Key decisions and trade-offs

Rules decide and AI suggests. The checks are deterministic, while the AI never blocks filing and a person accepts or dismisses each suggestion. The trade-off is that the AI still misses about half, which is why I report 13 of 27 and not the more flattering 17.

Checks are written against the schema, never against a document, so the same checks would run on the next client's packet. The trade-off is that five of the 15 families still depend on how free text is worded, so a client who phrases things differently could slip past them, which is where the AI reader is meant to help.

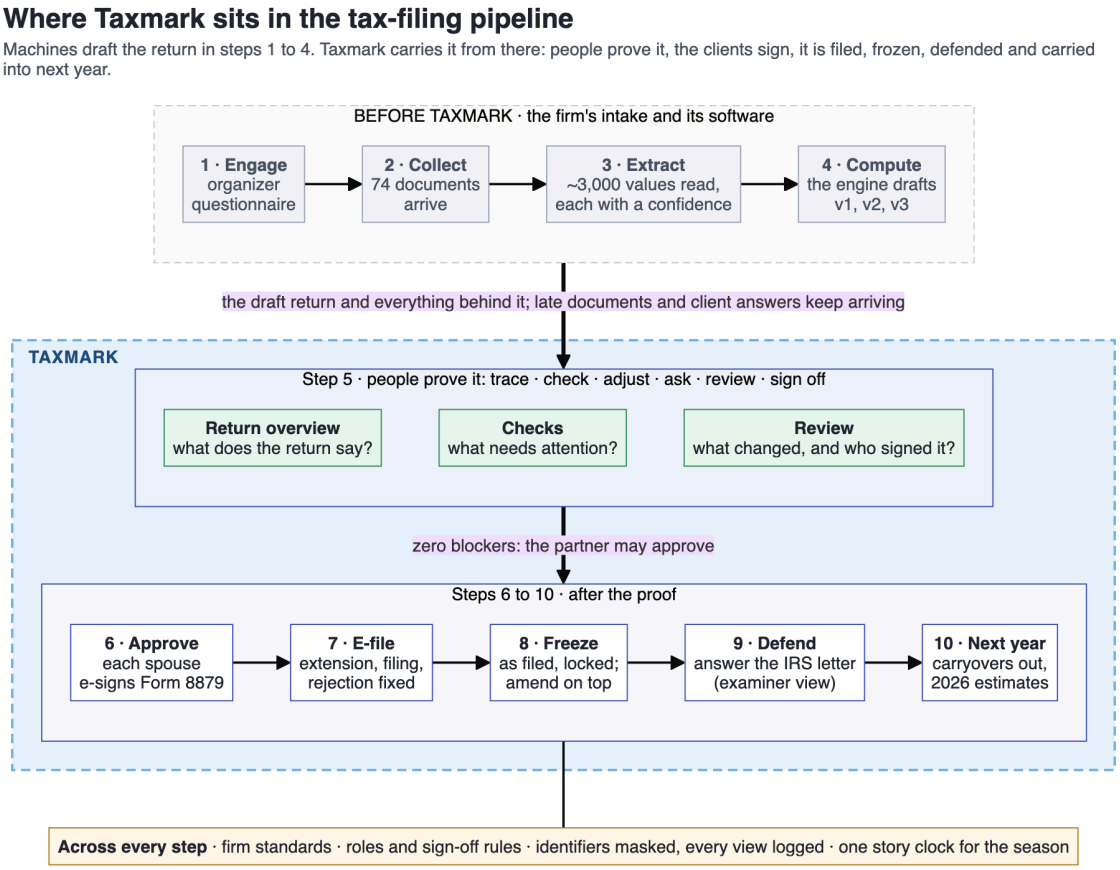
An event log over a read-only packet. History and undo come for free and nothing is ever lost. The trade-off is that every request replays the whole log, with each open tab asking again every five seconds, which is fine for one return but would need snapshots across a firm, and a mistake can only be reversed by a new event, never erased.

Honest flow-through. A fix carries through plain sums, so the refund moves the moment the payments are corrected. The trade-off is that a line needing a real tax computation keeps the engine's figure and blocks approval until the engine re-runs, and the tax effects shown are estimates from the packet's own rates.

A role switcher instead of sign-in, so one link shows every view while the server still enforces each role. The trade-off is that identity is self-declared: anyone with the link can switch to the partner and sign off, so real sign-in is the first thing to build before a firm uses it.

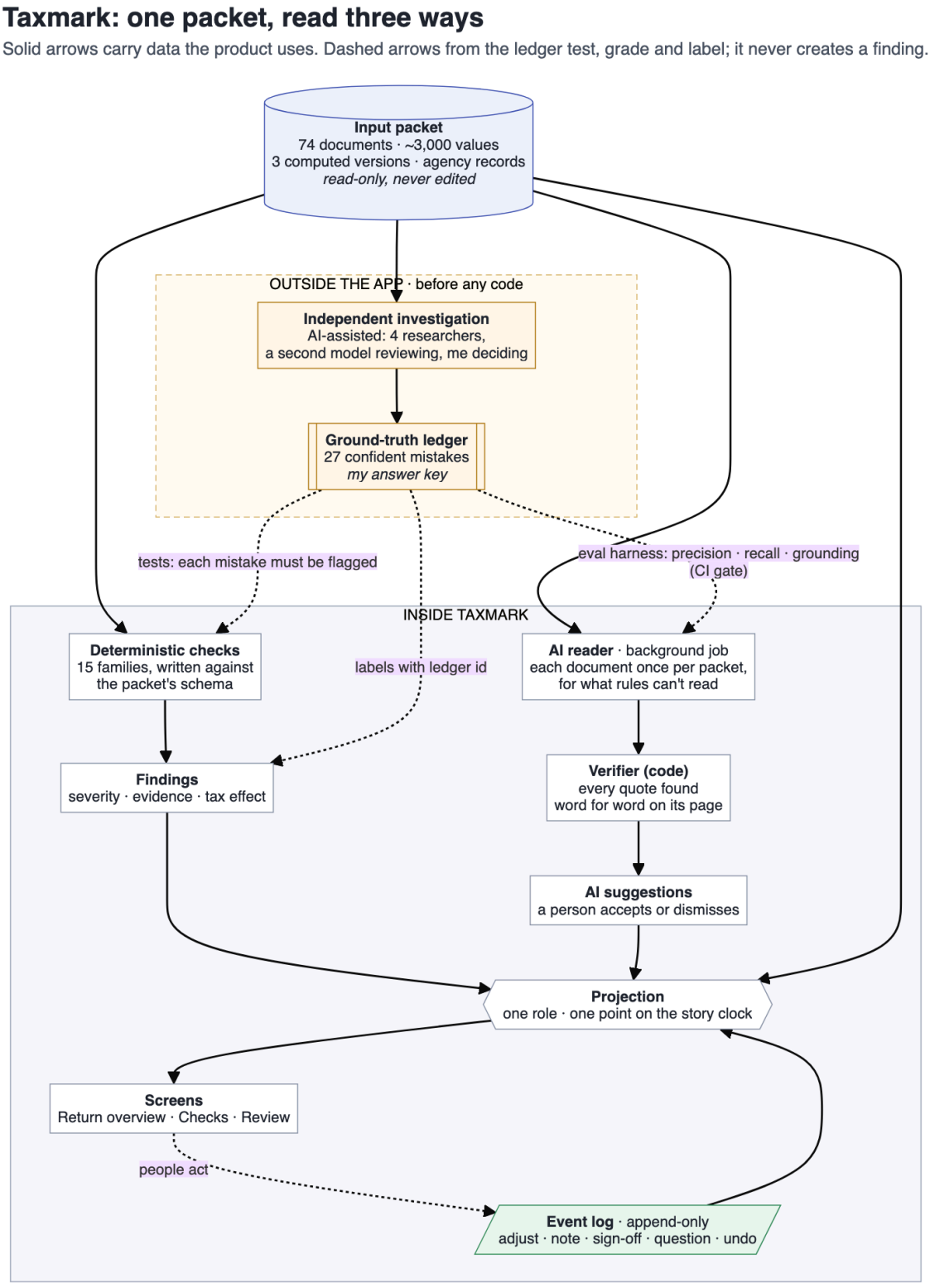
Appendix

A. Where Taxmark sits in the tax-filing pipeline



Steps 1 to 4 belong to the firm's intake and its software: the organizer, the documents, the extraction and the engine's draft. Taxmark picks the return up at step 5, where people prove it on three screens, each answering one question, and carries it through approval, e-filing, the frozen as-filed record, the IRS letter and into next year.

B. The data model



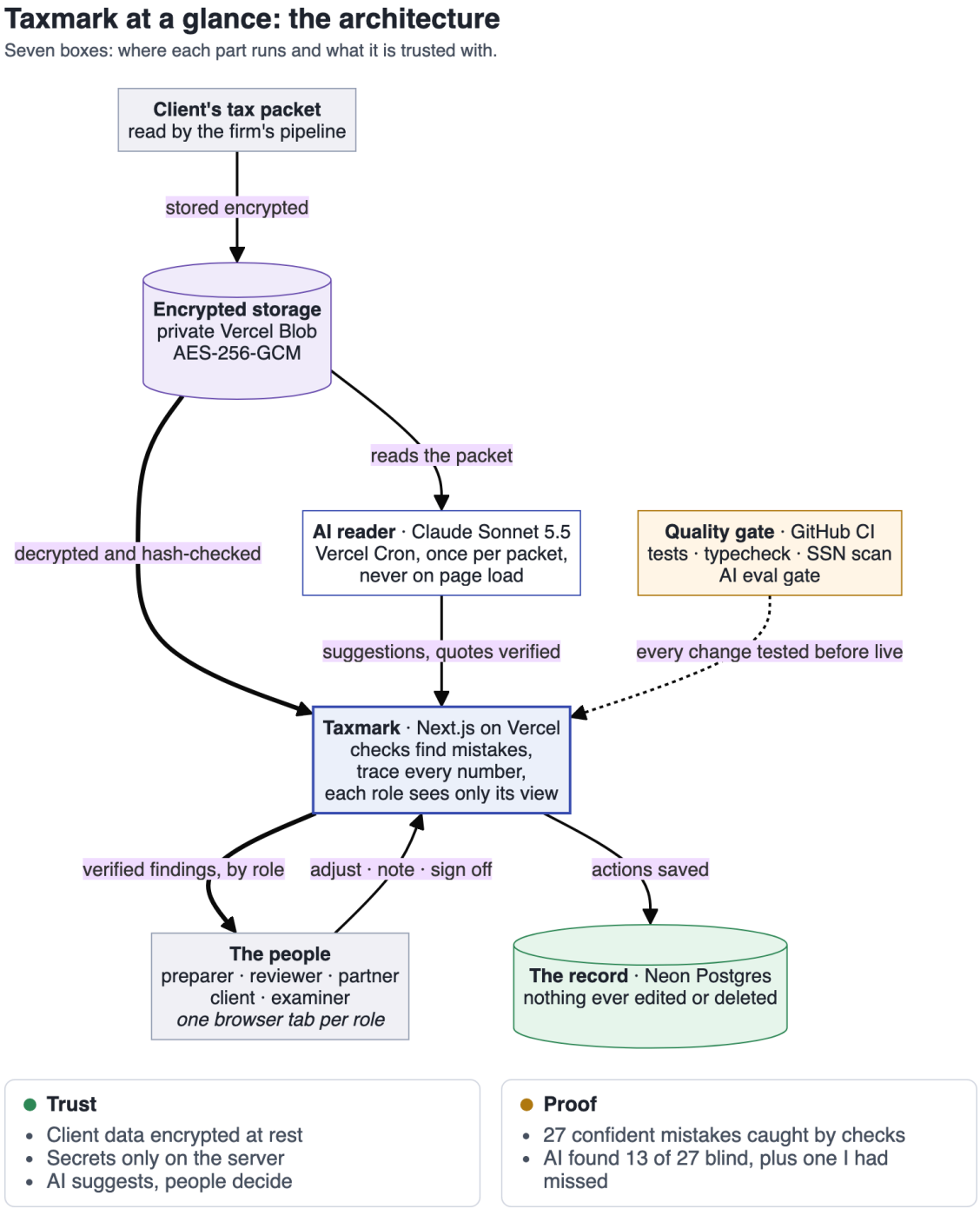
One packet, read three ways. The input packet was never split up or changed. The same packet was read by three readers, each with its own job, and that separation is what lets me trust the result:

- **The independent investigation (outside the app, before any code).** An AI-assisted hunt across the whole packet, with four AI researchers each taking one area, a second model reviewing their work, and me resolving anything in doubt, produced my ground-truth ledger: 27 distinct confident mistakes. This ledger is my answer key. It never creates a finding: the app only uses it to label a finding a check has already raised with its ledger id, and outside the app it tests the checks and grades the AI.
- **Deterministic checks (inside the app).** Each kind of mistake became a rule, 15 families in all, written against the packet's schema and never against a specific document, so the same checks would run on the next client's packet. There is a test for each of the 27 that fails if the app ever stops flagging it.
- **The AI reader (inside the app, for what rules can't read).** Some mistakes live in wording, like a client's answer or a note on a page, where rules are at their weakest. A background job runs an AI reader over every document once per packet, never while someone is browsing. Code then checks that every quote it gives is on its page, word for word, before a suggestion is shown, and a person accepts or dismisses each one.

Read-only underneath, everything on top. The packet and the AI reader's output stay read-only: never edited, only built on. Everything a person does sits on top of them:

- **Event log:** append-only. Adjustments, notes, sign-offs, questions, decisions, undo. Nothing is edited or deleted; an undo is a new event on top. The audit trail is the data, not a feature.
- **Projection:** for one role, at one point on the story clock: findings, blockers, sign-offs (fresh or stale), adjustments carried through the totals, masking.
- **Claim:** anything that can hold evidence and actions: a line, a document, a finding.

C. The architecture



Where each part runs. The packet is sealed with AES-256-GCM in a private Vercel Blob store and loaded by pointer, hash-checked, and refused if it does not match. A protected function on a daily Vercel Cron runs the AI reader (Claude Sonnet 5.5) with identifiers masked, and seals its output beside the packet. The Next.js app on Vercel runs the checks, the verifier and the projection on the server, enforces each role's policy and masks identifiers before anything reaches the browser. The event log lives in Neon Postgres, where the app only ever inserts and reads. GitHub Actions runs the unit tests (one per ledger mistake), a typecheck, an SSN scan and the AI eval gate, and the unit tests run again before every Vercel build.

D. How the AI reader was evaluated

Scored with a harness against the ledger (`docs/handin/ai-scorecard.md`):

Measure	Result
Grounding (every quote found on its page)	40 of 40 (100%)
Precision, same issue (judged by a second model)	25 of 40 (63%)
Recall, same issue (judged by a second model)	13 of 27 (48%)
Recall, automatic (any suggestion on the mistake's line)	17 of 27 (63%)
New real mistake not in the ledger	1

A test in CI fails the build if a suggestion is ungrounded or the automatic recall drops below 16 of 27.

E. The 27 confident mistakes

The full table, with the check that catches each one and its tax effect: `docs/handin/errors-found.md`.